

SISTEMAS OPERATIVOS Y REDES II

Segundo Semestre 2025

Trabajo Practico 4

Seguridad en HTTPS, Certificados y Autoridad Certificante

introducción

En esta serie de trabajos prácticos, los alumnos se sumergirán en la configuración y seguridad de servidores web, progresando desde la instalación y puesta en marcha de un servidor LAMP (Linux, Apache, MySQL, PHP) hasta la implementación de seguridad avanzada mediante HTTPS y certificados SSL. Cada trabajo práctico está diseñado para construir sobre los conocimientos del anterior, brindando una comprensión integral de los principios fundamentales de servidores web, seguridad de la información y autenticación basada en certificados.

Primera Parte: Instalación de Servidor LAMP

- Instalar y configurar un servidor web completo utilizando Linux, Apache, MySQL y PHP (LAMP).
- Asegúrese de que el servidor esté en funcionamiento, capaz de servir páginas web dinámicas.
- Comprender los componentes y su interacción dentro del servidor LAMP.
- Desarrollar habilidades básicas en la gestión de servidores Linux.

Segunda Parte: Configuración de Certificados y Seguridad en HTTPS

- Configure HTTPS en el servidor previamente instalado para asegurar la comunicación entre el cliente y el servidor.
- Implementar certificados SSL/TLS autofirmados en el servidor.
- Comprender la importancia del uso de HTTPS para garantizar la seguridad y la privacidad en las comunicaciones.
- Realizar pruebas de acceso seguro al servidor utilizando HTTPS.

Tercera Parte: Autoridades Certificadoras y Autenticación SSL

- Cree una autoridad certificadora (CA) y firme certificados digitales para autenticar las conexiones SSL.
- Configurar el servidor para usar certificados emitidos por una CA y comprender su funcionamiento.
- Entender el rol de las autoridades certificadoras en el ecosistema de seguridad web.
- Implementar y verificar la autenticación SSL en el servidor, utilizando certificados firmados por una CA.

Entrega:

1. Adjuntar código, archivos y capturas de pantalla que muestran el funcionamiento.
2. Entregar un informe que describa los pasos seguidos y las configuraciones realizadas.

Primera Parte

Instalación y configuración de un servidor LAMP

Objetivo:

El objetivo de este trabajo práctico es que los estudiantes instalen y configuren un servidor LAMP. Esta configuración les permitirá crear un servidor web con capacidades avanzadas para alojar y gestionar sitios web dinámicos utilizando bases de datos y lenguajes de programación del lado del servidor.

Consignas Generales:

1. **Entorno:** El trabajo debe realizarse en un servidor Linux. Puedes utilizar cualquier distribución basada en Debian (ej: Ubuntu).
2. **Paquetes necesarios:** Apache2, MySQL/MariaDB, PHP.
3. **Resultado esperado:** El servidor LAMP deberá estar completamente funcional y debe ser capaz de alojar al menos dos sitios web diferentes. Además, se debe instalar un CMS como WordPress (puede ser otro) y dejarlo listo para su uso.

Parte 1: Instalación de Apache**1. Instalar Apache:**

Utilice el siguiente comando para instalar Apache en su servidor:

```
sudo apt-get install apache2
```

Verifica su funcionamiento accediendo a la IP del servidor desde un navegador. Debes visualizar la página por defecto de Apache.

2. Configuraciones básicas de Apache:

Revise el archivo de configuración principal (apache2.conf) y realice las siguientes configuraciones:

- **Timeout:** Modifica el tiempo de espera del servidor a 300 segundos.
- **Puertos :** Revise el archivo ports.conf y asegúrese de que Apache esté escuchando en los puertos 80 (HTTP) y 443 (HTTPS).

3. Configuración de sitios virtuales:

Crea las carpetas para alojar dos sitios web diferentes:

```
mkdir -p /var/www/empresa1.com/public_html
mkdir -p /var/www/empresa2.com/public_html
```

Crea un archivo index.html en el directorio public_html de cada sitio. Ejemplo de archivo para **empresa1.com**:

```
<html>
<head>
  <title>Bienvenidos a Empresa1</title>
</head>
<body>
  <h1>has configurado correctamente Apache</h1>
</body>
</html>
```

4. Configuración de los archivos.conf :

- En el directorio /etc/apache2/sites-available, crea archivos de configuración para ambos sitios. Utilice el archivo 000-default.conf como plantilla.
- Configure los archivos para cada sitio, modificando los parámetros necesarios como ServerName y DocumentRoot.
- Activa los sitios utilizando los siguientes comandos:

```
sudo a2ensite empresa1.com.conf
sudo a2ensite empresa2.com.conf
sudo a2dissite 000-default.conf
sudo systemctl restart apache2
```

Parte 2: Instalación de MySQL y PHP

1. Instalar MySQL :

Instale MySQL con el siguiente comando:

```
sudo apt-get install mysql-server
```

Asegúrese de configurar la clave del usuario root de MySQL y ejecute configuraciones adicionales de seguridad con:

```
sudo mysql_secure_installation
```

2. Creación de base de datos y usuarios :

Inicia sesión en MySQL y crea una base de datos y un usuario:

```
mysql -u root -p
CREATE DATABASE dbprueba;
CREATE USER 'usuariotest'@'localhost' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON dbprueba.* TO 'usuariotest'@'localhost';
```

3. Instalar PHP :

Instale PHP y los módulos necesarios:

```
sudo apt-get install php libapache2-mod-php php-mysql
```

4. Probar PHP:

Crea un archivo info.php en el directorio de **empresa1.com** para probar la instalación de PHP:

```
<?php phpinfo(); ?>
```

Acceda a la URL <http://empresa1.com/info.php> desde un navegador para verificar la instalación de PHP.

Parte 3: Instalación de un CMS (WordPress)

1. Descarga e instalación de WordPress:

Descarga WordPress desde su página oficial y descomprime los archivos en el directorio `/var/www/empresa2.com/public_html`.

Asegúrese de que los archivos tengan los permisos adecuados para su funcionamiento.

2. Configuración de WordPress:

Acceda a la URL <http://empresa2.com/wp-admin/install.php> para iniciar el asistente de instalación de WordPress.

Complete los campos relacionados con la base de datos creada previamente y finaliza la instalación.

Segunda Parte

Configuración de Certificados y Seguridad en HTTPS

1. Introducción

En esta segunda parte del trabajo práctico, se explorarán técnicas de seguridad fundamentales para asegurar la comunicación en servidores web. Utilizaremos los certificados emitidos y autofirmados generados en la primera parte para implementar HTTPS en un servidor Apache. También se configurarán opciones de seguridad avanzadas como el uso de las mejores prácticas para el cifrado y la verificación de certificados.

2. Objetivos

- Configure un servidor Apache para soportar HTTPS.
- Implementar certificados autofirmados generados con OpenSSL.
- Probar y verificar la configuración HTTPS en un servidor web.
- Aplique las mejores prácticas de seguridad en la configuración del servidor.
- Analizar los registros y posibles errores en la implementación.

3. Desarrollo

3.1. Configuración de Apache para HTTPS

1. Habilitar los módulos necesarios en Apache

El servidor Apache necesita los módulos `ssl` y `socache_shmcb` habilitados para soportar HTTPS. Puedes activarlos ejecutando los siguientes comandos:

```
sudo a2enmod ssl
sudo a2enmod socache_shmcb
```

2. Configurar un VirtualHost para HTTPS

Crear una nueva configuración de un sitio habilitado para HTTPS en el archivo de configuración de Apache. Por ejemplo, cree el archivo `/etc/apache2/sites-available/secure-site.conf` con el siguiente contenido:

```
<VirtualHost *:443>
    ServerName secure-example.com
    DocumentRoot /var/www/html

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/alice_cert.pem
    SSLCertificateKeyFile /etc/ssl/private/alice.pem

    <Directory /var/www/html>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

3. Habilitar la configuración del sitio

Después de crear el archivo de configuración, habilite el nuevo sitio y recargue Apache:

```
sudo a2ensite secure-site.conf
sudo systemctl reload apache2
```

3.2. Generación y Uso de Certificados

1. Generación de un Certificado Auto-Firmado

Usaremos OpenSSL para crear un certificado auto-firmado, el cual se utilizará para el sitio configurado en el servidor Apache. Para generar el certificado, ejecuta:

```
openssl req -x509 -newkey rsa:2048 -keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt -days 365 -nodes
```

Asegúrese de proporcionar la información correcta para el Common Name (CN) servidor, ya que debe coincidir con el nombre del servidor (`ServerName`) configurado en Apache.

2. Configurar Apache para usar el Certificado Auto-Firmado

Actualiza el archivo de configuración del sitio (`/etc/apache2/sites-available/secure-site.conf`) para apuntar al nuevo certificado y clave generados:

SSLCertificateFile /etc/ssl/certs/selfsigned.crt
SSLCertificateKeyFile /etc/ssl/private/selfsigned.key

3.3. Prueba de configuración HTTPS

1. Reiniciar Apache

Luego de realizar los cambios en la configuración, reinicia el servidor Apache para que los cambios surtan efecto:

```
sudo systemctl restart apache2
```

2. Acceder al sitio web a través de HTTPS

Abra un navegador web y acceda al sitio utilizando la dirección IP o el nombre de dominio (si configurado) anterior https://. Es posible que reciba un aviso de seguridad porque el certificado está autofirmado. Acepta el riesgo para continuar y verificar que la página cargada correctamente.

3.4. Mejores Prácticas de Seguridad

1. Deshabilitar versiones antiguas de TLS y cifrados inseguros

Es recomendable deshabilitar versiones antiguas de TLS (como TLS 1.0 y 1.1) y evitar cifrados débiles. Para hacerlo, agregue la siguiente línea en la configuración del sitio:

```
SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1  
SSLCipherSuite HIGH:!aNULL:!MD5
```

2. Configurar HTTP Strict Transport Security (HSTS)

Para mejorar la seguridad de las conexiones HTTPS, puedes habilitar HSTS agregando la siguiente línea:

```
Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"
```

3. Revisar Logs y Diagnosticar Errores

Si experimenta problemas con la configuración, revise los archivos de registro de Apache /var/log/apache2/error.log y /var/log/apache2/access.log para encontrar posibles errores o advertencias.

Tercera parte

Autoridades Certificadoras y Autenticación SSL

En internet, es esencial contar con un conjunto de entidades certificadoras raíz en las que confía todo el mundo. Estas entidades certifican las llaves públicas de sitios web que necesitan demostrar su autenticidad a los usuarios. Las autoridades certificadoras (CA) raíz permiten que los usuarios de una web segura tengan la certeza de que están accediendo al sitio correcto y no a una falsificación de este.

En ocasiones, puede ser útil disponer de una entidad certificadora privada, fuera del árbol global de certificados. Esto resulta ventajoso en aplicaciones privadas donde se tiene control total tanto sobre el software cliente como sobre el servidor. Por ejemplo, una empresa que gestiona el acceso a edificios mediante tarjetas inteligentes no necesita que esas llaves estén firmadas por

una entidad externa. De manera similar, una empresa que distribuye actualizaciones de software a una plataforma hardware bajo su control puede optar por crear su propia entidad certificadora.

Objetivos

- Comprender la dinámica de funcionamiento de una autoridad certificadora (CA).
- Aprender a crear y revocar certificados.
- Implementar un sistema de autenticación basado en llaves asimétricas.

Desarrollo

3.1. Creación de una Autoridad Certificadora (CA) y Certificación de Llaves

En el trabajo práctico anterior, aprendimos a crear y firmar certificados. Estos conocimientos ahora se aplicarán en un entorno real con un servidor web, gestionado a través de una CA. Una CA no es más que una base de datos de certificados, con una llave privada y su certificado autofirmado correspondiente.

3.1.1. Despliegue del Árbol de Directorios y Configuración del CA

Los archivos relacionados con nuestra CA se almacenarán en un árbol de directorios. Para crear este entorno, ejecuta:

```
mkdir -p CA/certs CA/private CA/newcerts
```

El CA necesita inicialmente tres archivos clave:

- Un archivo de configuración que copiaremos de /etc/ssl/openssl.cnf.
- Un archivo de número de serie, que actúa como contador de los certificados firmados.
- Un archivo vacío que actuará como la base de datos del CA.

```
cp /etc/ssl/openssl.cnf CA/ca.conf
echo 1000 > CA/serial
touch CA/index.txt
```

En el archivo de configuración, especifica la ruta completa del directorio CA en la directiva dir(línea 42). Revise la política de firma en la sección [policy_match], que define los requisitos de las solicitudes que esta CA puede firmar.

Finalmente, protege el directorio private:

```
chmod 700 CA/private
```

Para tener el CA operativo, cree un certificado autofirmado que se utilizará para firmar las solicitudes de certificado recibidas por la CA:

```
openssl req -new -x509 -days 3650 -extensions v3_ca \
-keyout CA/private/cakey.pem -out CA/cacert.pem \
-config CA/ca.conf
```

Analice la información del certificado que acabas de crear. ¿Cómo puedes verificar que está autofirmado?

```
openssl x509 -in CA/cacert.pem -noout -text
```

Observe los cambios en el directorio CA. ¿Qué archivos se han creado o modificado?

3.1.2. Firma de una Solicitud de Certificado

Siguiendo la práctica anterior, crea una llave y la solicitud de firma correspondiente. Para evitar confusiones entre los roles de solicitante y CA, cree un directorio llamado llaves en la máquina anfitriona y dentro de este directorio genera una llave para "Carol".

Recuerda seguir la política de firma del CA vista anterior:

```
mkdir llaves; cd llaves
openssl genpkey -algorithm RSA -out carol-key.pem
openssl req -new -key carol-key.pem -out carol-req.pem
```

Envía la solicitud al CA que se encuentra en la máquina virtual:

```
scp carol-req.pem usuario@192...:
```

Como CA, recibe la solicitud y procede a firmarla:

```
openssl ca -config CA/ca.conf -out carol-cert.pem \
-infiles carol-req.pem
```

Envía el certificado firmado de vuelta al solicitante:

```
scp usuario@192...:carol-cert.pem .
```

Revisa los cambios en el árbol de directorios del CA. ¿Dónde se almacena el certificado de Carol? ¿Dónde queda la llave privada? Explora el nuevo certificado. ¿Dónde puedes ver la identidad del CA?

3.1.3. Importación de Llave y Certificado en Navegador

Para importar la llave y el certificado de Carol en Firefox, convierte estos archivos al formato PKCS12:

```
openssl pkcs12 -export -in carol-cert.pem \
-inkey carol-key.pem -out carol.p12
```

3.2. Autenticación de Clientes en Apache con SSL

Una vez creado el certificado de Carol, configuraremos Apache para que solo permita acceso a usuarios que presenten un certificado emitido por nuestra CA. Modifica el archivo 001-default-ssl cambiando la línea:

```
SSLVerifyClient require
```

Después de reiniciar Apache, intenta acceder mediante HTTPS. ¿Qué errores se producen? ¿A qué se deben?

Lee los comentarios en el archivo de configuración y realiza los ajustes necesarios para establecer una conexión HTTPS correctamente.

3.3. Revocación de Certificados Emitidos por la CA

Los certificados tienen fechas de validez, pero en caso de comprometerse, una CA puede revocarlos. Para esto, utilice una **Lista de Revocación de Certificados (CRL)**.

3.3.1. Creación de una CRL Inicial

Crea un directorio crly el archivo de número de CRL:

```
mkdir CA/crl  
echo 1000 > CA/crlnumber  
openssl ca -config CA/ca.conf -gencrl -out CA/crl.pem
```

Examina los cambios en el directorio CA. ¿Por qué fue necesario ingresar la clave privada para generar la CRL? Visualiza la CRL con el comando:

```
openssl crl -in CA/crl.pem -noout -text
```

3.3.2. Revocación de un Certificado

Si se compromete la seguridad de un certificado, puede revocarse:

```
openssl ca -config CA/ca.conf -revoke CA/newcerts/1000.pem
```

¿Qué cambios ocurren en el archivo index.txt? ¿Se ha actualizado la CRL?

3.3.3. Publicación de una CRL Actualizada

Después de revocar un certificado, actualice la CRL:

```
openssl ca -config CA/ca.conf -gencrl -out CA/crl.pem
```

Examina el archivo crl.pem. ¿Qué información contiene sobre los certificados revocados?

3.4. Configuración de CRL en Apache

Aunque hemos revocado el certificado de Carol, todavía puede autenticarse porque Apache no está verificando la CRL. Modifique el archivo de configuración de Apache para que lea la CRL y evite conexiones con certificados revocados